



THE CITY OF VAEL

DOORS & LOCKS

03 – Auth and RLS Review

Who can enter the City today, and what the locks on the Foundation drawing would do if they were installed.

PREPARED FOR

Tony Grant, Owner
TSG Construction LLC

DOCUMENT TYPE

Milestone 1 deliverable
PRD acceptance pack

PREPARED BY

Appifest – Technical Audit
Continuing the existing prototype

DATE / METHOD

25 August 2026
Read-only inspection of the live tree

1. Two locksets

Doors & Locks means authentication, permissions, and row-level security. The prototype has a working front door (sign up, sign in, session). The Foundation drawing has a much stricter lockset (RLS policies, roles, function-only writes). They are not the same lockset. This inspection did not execute RLS against a live database – none is attached to the running app. What follows is the prototype path as coded, and the drawing as written.

2. The door people actually use

`useAuth` → `authService` → by default `mockAuthProvider` wrapping `mockAuth.ts`. Accounts and session live in `localStorage`. Password reset is simulated. A “verify email” control in development calls `simulateVerifyEmail()`.

The Supabase auth provider is implemented, not stubbed: sign up, sign in, sign out, session, password reset, resend verification. It is **inert** unless both of these are true:

1. Someone sets `VITE_AUTH_PROVIDER=supabase` on purpose (not merely because env vars exist).
2. `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` are present so the client is not `null`.

No `.env` exists in this tree. The product README still mentions `.env.example`; that file is **missing** from this copy. Default mode is mock. That dual-flag rule is good Doors discipline – the City will not silently switch production authority because a URL happened to be in the environment.

3. What would break if the live door were swapped today

The session seam itself would keep working: `user.id`, email, sign-in/out. The break is every screen that still asks `mockAuth` for identity using that id. A real Auth uuid is not in the mock account book.

DOOR / ROOM	CALL	FAILURE
Handle claim gate	<code>mockAuth.getProfile(user.id)</code>	Null for every real user – username gate breaks
Claim handle dialog	availability / claim on mock store	Checks the wrong book
Join network	<code>updateProfile(user.id, { profileType })</code>	Onboarding type write no-ops
Waitlist form	<code>getUserById</code>	Lookup fails

DOOR / ROOM	CALL	FAILURE
Team tab	<code>searchAccounts</code>	Searches the mock list, not real users
Admin users / badges	<code>getAllAccountsForAdmin</code>	Shows mock accounts, not the real roster
Dev nav admin toggle	<code>simulateGrantAdmin</code>	Unrelated to <code>user_roles</code>

Broader grep: dozens of files still reach `mockAuth` for directory/enrichment. Same id-mismatch class of bug. Auth is the one seam that *is* wired – which makes leftover direct calls more dangerous than in VAE/Connection, where flipping the flag currently changes nothing because screens never call those services.

4. Admin is a flag, not a role, in the prototype

Mock mode: `isAdmin` boolean, flipped by a development simulate function. Foundation drawing: six roles on `user_roles` – **owner, admin, manager, developer, moderator, member**. Last remaining Owner cannot be revoked. Grants go through security-definer functions, not a column the user can update on themselves.

The Supabase auth provider hardcodes `isAdmin: false` and says so in its own comments: reading real roles needs a round-trip this provider does not perform. Even a true Owner account would look like a member in the UI after a naive cutover. There is no `src/lib/governance/` module. Moderation RPCs require owner/admin/moderator exactly – manager and developer are excluded. The moderation service has **no local provider**; without a live Foundation it refuses every operation.

5. Handshake as a lock, not only a Room

The PRD: full profiles revealed only after both parties accept. In the prototype, connection rows are party-private in practice because they live in that browser’s storage and the UI only shows “your” connections. On the Foundation drawing, `connections` SELECT is party-only; lifecycle fields are not generally updatable – accept/decline/connect/expire go through named functions (0006). That is the correct lock for a Handshake Room. It is drawn, not installed on the running app.

Messages: participant-only SELECT; send via `send_message()`. Blocks: blocker-only SELECT; the blocked party has no row access. Reports: reporter or staff; the target does not read the report. These are strong locks on paper.

6. The open (now closed) profile door

`profiles` (username, display name) is publicly readable. `district_profiles` (type + JSON) is authenticated-only. Migration 0019 records an Owner decision: that authenticated-only rule stays. Anonymous visitors get no table privilege on `district_profiles`.

The prototype `ProfilePage` still displays profile details to anonymous visitors by reading `mockProfiles` directly. The directory service already withholds district profile data when there is no viewer id. So the City currently has two answers to the same door: the new directory lock matches the drawing; the old details pane does not. That is why profile *writes* were migrated and profile *reads* were deliberately left on the old path – migrating the read would silently change visitor visibility. Closing it is a product decision about the door, then a small Framing change, not a surprise lock change in SQL.

DIRECTORY SAFETY

Public directory types exclude email, phone, and isAdmin. On the Foundation, those columns are not on `profiles` at all (structural). In local mode, safety depends on one adapter discarding `MockUser` private fields (conventional). Currently correct; weaker than the SQL shape.

7. RLS and privilege hardening (as drawn)

PostgreSQL checks table privileges *before* RLS. 0001–0016 wrote careful row policies, then revoked some rights, but did not always grant the base rights those revokes assumed. Local CLI stacks and hosted Supabase projects start from opposite defaults. 0017 exists to force one end state. 0020 then strips `service_role` blanket table DML so privileged work must go through security-definer functions. This audit confirms those files exist and say that; it does not certify they have been applied anywhere this app is running.

0018 fixes a real locksmith defect: `ensure_conversation_for_connection()` used an `ON CONFLICT` column list that was ambiguous with an output parameter, which would have failed the first time the function ran. 0019 adds a narrow owner-only function to read document admin notes without widening column grants – so a rejected document owner can see what to fix, without handing internal notes to the public.

8. SMS / marketing consent locks

`sms_consents.phone_verified` has no legitimate writer in the drawing. Transactional and marketing consent go through dedicated functions. The UI labels non-in-app channels **NOT YET CONNECTED**. Consent records can exist locally; no message is sent. That is honest. Connecting Twilio or similar is a Change Order and a new lock review, not a toggle.

9. Doors & Locks verdict

DO NOT SWAP THE FRONT DOOR YET

The prototype door works for a single-device demo. The Foundation lockset is substantially better – and currently unreachable. Cut over auth alone, and Handshake, handle claim, admin, and onboarding break. Cut over nothing until identity/directory either moves through the seam or a bridge maps Auth ids onto the remaining mock stores. Owner/Admin in the UI must read `user_roles`, not a boolean, before anyone treats the City as a real multi-user building.